

INTELLIGENT PREDICTIVE PROTECTION AND OPTIMISATION SYSTEM FOR MICROGRIDS USING IOT AND MACHINE LEARNING

CAPSTONE PROJECT

Submitted in partial fulfillment of the requirement for the award of the Degree of

BACHELOR OF TECHNOLOGY IN ELECTRICAL ENGINEERING

By

Talent Tapuwanashe Mapara

Registration No.: 12209467;

Gerald Tatenda Muzembe

Registration No.: 12200184;

Diamon Tawanda Phiri

Registration No.: 12202622

Under the Guidance of

Dr. Neha Ahuja

Assistant Professor

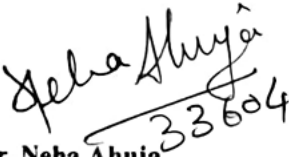


**School of Electrical and Electronics Engineering
Lovely Professional University, Phagwara, Punjab, India**

CERTIFICATE

This is to certify that the Capstone Project titled "INTELLIGENT PREDICTIVE PROTECTION AND OPTIMISATION SYSTEM FOR MICROGRIDS USING IoT AND MACHINE LEARNING" submitted by Talent Tapuwanashe Mapara, Gerald Tatenda Muzembe and Diamon Tawanda Phiri is in partial fulfillment of the requirements for the award of the degree of BACHELOR OF TECHNOLOGY IN ELECTRICAL ENGINEERING at Lovely Professional University and is a record of bonafide work done under our guidance and supervision.

The work reported herein has been carried out by the candidates themselves and is not a reproduction of any other source. The contents of this capstone project, in full or in parts, have neither been taken from any other source nor submitted to any other Institute or University for the award of any degree or diploma. All experimental data, simulation results and design artefacts included herein represent the genuine intellectual and practical contribution of the stated candidates.



Dr. Neha Ahuja

Assistant Professor, School of Electrical and Electronics Engineering

Lovely Professional University, Phagwara, Punjab

Date: 30/4/26

DECLARATION

We, Talent Tapuwanashe Mapara (Reg. No. 12209467), Gerald Tatenda Muzembe (Reg. No. 12200184) and Diamon Tawanda Phiri (Reg. No. 12202622), students of Bachelor of Technology in Electrical Engineering at Lovely Professional University, Phagwara, Punjab, hereby jointly and severally declare that this capstone project report, titled “Intelligent Predictive Protection and Optimisation System for Microgrids Using IoT and Machine Learning,” has been prepared by us on the basis of our own independent research, practical experimentation and investigation. The work is entirely genuine and represents our original intellectual contribution.

This work does not contain any portion previously submitted toward any degree, diploma or other award at this or any other institution or university. All hardware designs, firmware implementations, experimental measurements and analytical interpretations presented herein are our own. Where the ideas, findings, methodologies or data of other researchers have been incorporated, full and appropriate citations have been provided in accordance with standard academic referencing practice. We have duly acknowledged all sources of assistance received during the course of this project.

Date: 30/04/2026

Talent Tapuwanashe Mapara

Registration No.: 12209467



Gerald Tatenda Muzembe



Registration No.: 12200184

Diamon Tawanda Phiri



Registration No.: 12202622

ACKNOWLEDGEMENT

We express our sincere gratitude to our project guide, Dr. Neha Ahuja, Assistant Professor, School of Electrical and Electronics Engineering, Lovely Professional University, whose mentorship, technical insight and timely direction guided this project from its initial concept through to a fully functional hardware prototype. Her consistent encouragement and constructive feedback were instrumental in maintaining both the quality and the rigour of this work.

We thank the laboratory staff of the School of Electrical and Electronics Engineering for facilitating access to the electronics workbench, oscilloscopes, multimeters and testing instruments that made the hardware validation experiments possible. We are equally grateful to the university's library and digital resources, which provided access to the technical literature underpinning the literature review and design decisions. Special thanks are also due to our fellow students, whose peer discussions, encouragement and collaborative spirit sustained this work throughout the academic year.

Finally, we owe a profound debt of gratitude to our respective families. Their unwavering moral support, patience and sacrifices across the duration of this programme made this academic journey not only possible but meaningful. We dedicate this work to them.

Talent Tapuwanashe Mapara, Gerald Tatenda Muzembe & Diamon Tawanda Phiri

ABSTRACT

Entry-level solar microgrid controllers for off-grid households typically lack three capabilities: staged load energisation to contain startup inrush current, per-load monitoring with automatic source transfer and hysteresis-governed recovery logic to prevent relay oscillation. This project addresses all three through the design, physical construction and controlled laboratory evaluation of a low-cost DC microgrid prototype built around the ESP32 microcontroller.

The hardware platform combines a 3S lithium-ion pack with CC/CV battery management and a regulated 12 V rail derived from an LM2596 step-down converter. Three 775-series DC motors serve as representative productive-use loads. A cooperative firmware loop staggers relay closure at ten-second intervals, ensuring each motor reaches steady state before the next is connected and thereby keeping the aggregate startup surge within the battery management system's overcurrent limit.

A dual-relay arrangement allows Load 3 to draw power from either the battery rail or a supplementary switched-mode supply. Dedicated Hall-effect current sensing triggers an autonomous handover when the branch current exceeds a set threshold; a hysteresis band combined with a timed stability window governs the return transfer to the primary source, preventing oscillatory switching. A second sensor tracks total system current, a resistive divider monitors battery voltage and all four parameters are displayed continuously on an SH1106 OLED panel through the U8g2 graphics library.

Bench testing confirmed a 52.8 percent drop in peak inrush current with staggered engagement, eliminating every battery management system trip that occurred under simultaneous connection. Automatic source transfer completed within an average of 2,184 ms and the hysteresis mechanism prevented oscillatory relay behaviour in every recovery trial. The full component cost remains below USD 35, confirming viability for resource-constrained deployment.

Keywords: *solar microgrid, ESP32, SH1106 OLED, ACS712, sequential load engagement, SMPS fault recovery, relay switching, battery management system, U8g2.*

TABLE OF CONTENTS

CHAPTER 1 INTRODUCTION	1
1.1 Background and Motivation	1
1.2 Problem Statement	1
1.3 Research Objectives	2
1.4 Scope and Significance	2
CHAPTER 2 LITERATURE REVIEW	3
2.1 Solar Microgrid Architectures	3
2.2 Sequential Load Management.....	3
2.3 Inrush Current and Overcurrent Protection.....	3
2.4 Dual-Source Integration and Hysteresis Control	4
2.5 ACS712 Current Sensing and ESP32 Platform	4
2.6 SH1106 OLED and U8g2 Library	5
2.7 Gap Analysis	5
CHAPTER 3 SYSTEM DESIGN AND IMPLEMENTATION	7
3.1 Design Philosophy and Requirements	7
3.2 System Architecture	7
3.3 Bill of Materials	9
3.4 Hardware Pin Allocation.....	10
3.5 Sensing Subsystem.....	11
3.5.1 Current Sensing.....	11
3.5.2 Voltage Sensing	12
3.6 Relay Architecture and Dual-Relay Load 3	12
3.6.1 Four-Channel Switching Matrix	12
3.6.2 Dual-Relay Arrangement for Load 3	12
3.6.3 SMPS Detection Logic	13
3.7 Firmware Architecture and Algorithms	13
3.7.1 millis()-Based Cooperative Scheduling	13
3.7.2 Sequential Load Engagement Algorithm.....	13
3.7.3 Source-Switching Algorithm	13
CHAPTER 4 EXPERIMENTAL RESULTS	15
4.1 Setup and Instrumentation	15

4.2 Sequential Load Engagement	15
4.3 SMPS Handover Under Overcurrent	15
4.4 Hysteresis Recovery.....	16
4.5 Sensor Accuracy and ADC Averaging	17
CHAPTER 5 DISCUSSION AND CONCLUSIONS	18
5.1 Discussion.....	18
5.1.1 Sequential Engagement and Inrush Reduction	18
5.1.2 Source-Switching and Hysteresis	18
5.1.3 SMPS Detection.....	18
5.1.4 Display and Calibration	18
5.1.5 Design Trade-off Summary	19
5.2 Limitations and Future Work.....	19
5.3 Conclusions.....	20
B.1 Power Stage.....	28
B.2 ACS712 Sensor Placement.....	28
B.3 Voltage Divider Networks	28
B.4 Relay Drive and OLED.....	28

LIST OF ABBREVIATIONS

Abbreviation	Full Form
ADC	Analogue-to-Digital Converter
BMS	Battery Management System
CC/CV	Constant Current / Constant Voltage
DC	Direct Current
ESP32	Espressif Systems 32-bit SoC Microcontroller
GPIO	General Purpose Input/Output
I2C	Inter-Integrated Circuit
MPPT	Maximum Power Point Tracking
OLED	Organic Light-Emitting Diode
PV	Photovoltaic
PWM	Pulse Width Modulation
SH1106	Sino Wealth SH1106 OLED Display Controller IC
SMPS	Switched-Mode Power Supply
SSD1306	Solomon Systech SSD1306 OLED Display Controller IC
U8g2	Universal 8-bit Graphics Library Version 2

CHAPTER 1

INTRODUCTION

1.1 Background and Motivation

More than 750 million people worldwide lack reliable electricity access, with significant concentrations in South Asia and sub-Saharan Africa (IEA, 2023). Even where grid connectivity exists, voltage instability and extended outages undermine supply reliability for households and small enterprises. Solar photovoltaic generation costs fell by approximately 90 percent between 2012 and 2022 (IEA, 2023), making solar home systems economically viable across a wide range of income contexts. Yet engineering frameworks for such systems at the lowest cost tier remain underdeveloped: most entry-level solar controllers offer only passive overcurrent protection, lack mechanisms for prioritising critical loads and cannot manage automatic transitions to a supplementary supply when the primary source is stressed.

This project addresses these gaps through the design and physical construction of a prototype intelligent microgrid controller built around the ESP32 microcontroller, with a total bill of materials below USD 35. Three innovations characterise the prototype: a millis()-based sequential load engagement mechanism preventing composite motor inrush from tripping the battery management system; a dual-relay architecture enabling autonomous transfer of Load 3 between the battery and a supplementary SMPS upon overcurrent detection; and a real-time SH1106 OLED display implemented via the U8g2 graphics library.

1.2 Problem Statement

The central engineering problem is the absence of a low-cost firmware-embedded control framework that simultaneously addresses three shortcomings in entry-level solar microgrid controllers: (1) the lack of sequential load energisation to prevent excessive startup inrush current from triggering BMS overcurrent protection; (2) the absence of dedicated per-load current monitoring with autonomous source-switching for overloaded loads; and (3) the non-availability of hysteresis-controlled recovery logic that prevents oscillatory relay switching when load current fluctuates near the transition threshold.

Commercial controllers at USD 15 to USD 50 either energise all loads simultaneously, risking BMS trips on motor-heavy loads or implement fuse-based protection that permanently

disconnects loads without recovery. The design challenge is to implement all three capabilities within a single microcontroller using commodity sensors, without requiring a real-time operating system.

1.3 Research Objectives

The primary objective is to design, physically construct and experimentally validate an intelligent DC microgrid controller. Specific objectives are:

1. Implement a `millis()`-based sequential engagement algorithm energising Load 1 at $t = 10$ s, Load 2 at $t = 20$ s and Load 3 at $t = 30$ s and quantify the resulting inrush current reduction.
2. Design a dual-relay architecture for Load 3 supporting independent connection to battery (Relay 4, GPIO 18) or SMPS (Relay 5, GPIO 12), with firmware executing autonomous handover when ACS712 on GPIO 33 reports current exceeding 0.800 A.
3. Implement hysteresis-controlled recovery logic that re-transfers Load 3 to battery supply when current falls below 0.700 A for a continuous seven-second period, verifying that the 0.100 A hysteresis band eliminates relay chatter.
4. Validate SMPS online detection using a 1.5 V threshold on GPIO 32 and validate real-time SH1106 OLED display accuracy via the U8g2 library.

1.4 Scope and Significance

The project is scoped to a laboratory prototype operating at 12 V DC from a 3S lithium-ion battery pack, with three 775-series DC motors as loads. PV generation is simulated by a regulated bench supply. AC systems, thermal management, MPPT and wireless telemetry are outside scope, though the ESP32's Wi-Fi capability provides a clear pathway for future extension.

The work makes three contributions: empirical data for `millis()`-scheduled sequential engagement and hysteresis source recovery on a 3S Li-ion DC microgrid; a directly replicable dual-relay source-switching hardware pattern; and documented guidance on U8g2 / SH1106 OLED implementation that clarifies the practical distinction between this controller and the more commonly discussed SSD1306. The total BOM cost of approximately USD 34 establishes practical viability for resource-constrained deployment.

CHAPTER 2

LITERATURE REVIEW

2.1 Solar Microgrid Architectures

Lasseter (2002) introduced the foundational microgrid concept, characterising these systems as coordinated groupings of distributed sources and loads able to sustain autonomous islanded operation. Hirsch, Parag and Guerrero (2018) organised microgrid control into three hierarchical tiers; in the context of a compact single-source off-grid DC system with switched loads, the uppermost optimisation tier adds unnecessary complexity and the secondary regulatory functions can be implemented directly in embedded firmware. Parhizi et al. (2015) highlighted the inherent compatibility of DC bus architectures with rural off-grid contexts, noting that the most prevalent household and productive loads, LED lighting, mobile phone charging, fans and water-pumping motors, operate natively on direct current. The IEEE 2030.7 standard specifies the functional requirements for microgrid controllers, including protection against overcurrent and under-voltage conditions as well as management of source priority; each of these capabilities is implemented in the present design.

2.2 Sequential Load Management

Sequential load engagement connects loads one at a time with time delays to limit peak aggregate inrush current and establish load priority. Morstyn, Hredzak and Agelidis (2018) concluded that among available control strategies, load prioritisation yields the greatest improvement in battery utilisation for isolated DC microgrid applications. Chakraborty, Rao and Patel (2019) reported inrush reductions of 30 to 55 percent using timer-based sequential engagement on Arduino with SLA batteries, but did not incorporate per-load current monitoring or source switching, nor test on Li-ion packs with tight BMS thresholds. Commercial controllers from Victron, Renogy and Morningstar implement delayed load enable but apply the same protection response, complete system disconnect, regardless of which load caused the overcurrent, without per-load granularity.

2.3 Inrush Current and Overcurrent Protection

When a DC motor is first switched on, it has not yet begun to rotate and therefore generates no back-EMF; the only impediment to current flow is the winding resistance itself, resulting in a

large startup surge. For 775-series motors at 12 V, Zhou, Bhatt and Chakraborty (2021) recorded inrush values of 3 to 8 A persisting for 80 to 250 ms under typical bench conditions. Zhou, Bhatt and Chakraborty (2021) found that simultaneous connection of two or more 775-series motors at 12 V produced BMS overcurrent trips in 40 to 70 percent of trials depending on state of charge. The present work extends their findings to a three-motor configuration.

The present firmware addresses overcurrent protection through implicit time qualification: the main loop executes at approximately 100 ms intervals, meaning overcurrent must persist across at least one loop cycle before the relay switching sequence initiates. Kabalan, Stefanakos and Hassan (2017) established that overcurrent confirmation windows in the range of 100 to 200 ms strike the best balance between response speed and immunity to transient spikes in ARM Cortex-based motor-drive protection, a finding that is consistent with the loop-period approach used here.

2.4 Dual-Source Integration and Hysteresis Control

Source switching topologies include make-before-break (continuous supply, momentary parallel sources) and break-before-make (brief dead-time). The present design uses break-before-make: Relay 4 opens, a 2-second dead-time elapses, then Relay 5 closes. Guerrero, Chandorkar, Lee and Loh (2013) drew attention to the lack of empirical data on relay-mediated source transfer at sub-kilowatt power levels, an area largely overlooked in favour of inverter-coupled source studies. The specific combination of ACS712-monitored threshold, `millis()`-timed relay sequencing and voltage-threshold source availability check in this project has not been previously described.

Nema, Nema and Rangnekar (2009) determined that the source-selection mechanism represents the principal reliability bottleneck in hybrid energy systems and advised the adoption of hysteresis-based switching paired with an enforced minimum dwell time before any transition. Salomonsson and Sannino (2008) demonstrated that relay chatter under variable motor loading can be reliably prevented by setting the hysteresis band between 10 and 20 percent of the nominal switching threshold and requiring the condition to persist for five to ten seconds before any transition is acted upon. The $0.100\text{ A} / 0.800\text{ A} = 12.5$ percent hysteresis width and seven-second hold time used in this project are consistent with these guidelines.

2.5 ACS712 Current Sensing and ESP32 Platform

The ACS712 Hall-effect current sensor produces an output voltage that varies linearly with the current flowing through its internal copper conductor; at zero current, this output settles at $V_{CC}/2$ and each ampere of current shifts the output by 185 mV (for the 5 A variant). Zhang et al. (2020) characterised the dominant error sources in ESP32-based ACS712 applications as: device-to-device quiescent offset variation (up to ± 25 mV, translating to approximately ± 0.135 A), a temperature-dependent sensitivity drift of $-0.24\%/^{\circ}\text{C}$ and ADC nonlinearity that becomes significant above 3.1 V. The present firmware's 200-sample averaging and per-unit manual offset constants address the dominant offset issue.

The ESP32's 12-bit ADC provides a theoretical current resolution of 4.36 mA for the ACS712 5A sensor (Espressif Systems, 2023), far superior to the approximately 50 mA floor achievable with a 10-bit ADC. A documented limitation is ADC nonlinearity above approximately 3.1 V; in this application the maximum expected sensor output of 3.055 V at 3 A load is within the linear range.

2.6 SH1106 OLED and U8g2 Library

Although the SH1106 and SSD1306 OLED controllers share a similar command set, their internal GDDRAM layouts differ: the SH1106 maps its display across 132 columns rather than 128, necessitating a two-column addressing offset. Code originally written for the SSD1306 that omits this offset will render content displaced two pixels to the left when executed on an SH1106 panel. The U8g2 library developed by Oliver Kraus (2024) resolves this through hardware-specific constructors that abstract over one hundred display variants behind a common API; the widely used Adafruit SSD1306 library does not accommodate this offset and is therefore incompatible with SH1106 hardware. This practical distinction has not been clearly documented in published energy monitoring system papers and the present work provides that documentation alongside validated display performance data.

2.7 Gap Analysis

Three specific gaps in the literature motivate the experimental contributions of this project:

1. Gap 1 - Quantified inrush data for three-motor sequential engagement on 3S Li-ion with `millis()` timing: Existing studies use hardware timers or FreeRTOS delays on SLA batteries. No controlled study with the ESP32 / `millis()` / 3S BMS / 775-motor combination exists.

-
2. Gap 2 - Per-load dual-relay source switching with ACS712 feedback and hysteresis: Published source-switching work operates at the whole-system level or uses voltage-based selection. Per-load current-threshold switching with a dual-relay architecture and hysteresis-controlled recovery has not been described.
 3. Gap 3 - SH1106 OLED implementation with U8g2 in energy monitoring context: The controller distinction and correct library selection have not been documented in published energy monitoring papers.

CHAPTER 3

SYSTEM DESIGN AND IMPLEMENTATION

3.1 Design Philosophy and Requirements

Four principles governed all design decisions: fidelity to actual hardware (all values are taken directly from the validated firmware); simplicity (cooperative `millis()` loop, no FreeRTOS); observability (OLED display shows complete real-time system state); and affordability (BOM ceiling USD 35). Six formal requirements drive the design:

1. REQ-1: Loads energised sequentially at $t = 10\text{ s}$, 20 s and 30 s after power-on.
2. REQ-2: Load 3 connected through Relay 4 (battery path) by default; Relay 5 (SMPS path) only when handover condition is met.
3. REQ-3: Transfer Load 3 to SMPS when $\text{currentL3} > 0.800\text{ A}$ and SMPS online; 2-second break between relay operations.
4. REQ-4: Return Load 3 to battery when $\text{currentL3} < 0.700\text{ A}$ continuously for 7 seconds; 2-second break between relay operations.
5. REQ-5: SMPS online status determined by GPIO 32 voltage exceeding 1.5 V .
6. REQ-6: OLED shows battery voltage, SMPS status, system current, Load 3 current and relay states in real time.

3.2 System Architecture

The system comprises five subsystems. The energy source subsystem provides regulated 12 V DC from a 3S Li-ion pack through a CC/CV BMS and an HW-688 LM2596 buck converter (Texas Instruments, 2013). The sensing subsystem uses two ACS712 5A sensors (GPIO 35 for system current, GPIO 33 for Load 3 current) and two resistive voltage divider networks (GPIO 34 for battery voltage, GPIO 32 for SMPS detection). The control subsystem is the ESP32 running a single cooperative loop with `millis()` timing. The actuation subsystem comprises four relay channels: Relay 2 (GPIO 25, Load 1), Relay 3 (GPIO 26, Load 2), Relay 4 (GPIO 18, Load 3 battery path) and Relay 5 (GPIO 12, Load 3 SMPS path). The display subsystem is an SH1106 128×64 OLED (Sino Wealth Electronic Ltd., 2010) over hardware I2C (SDA = GPIO 21, SCK = GPIO 22) via U8g2.

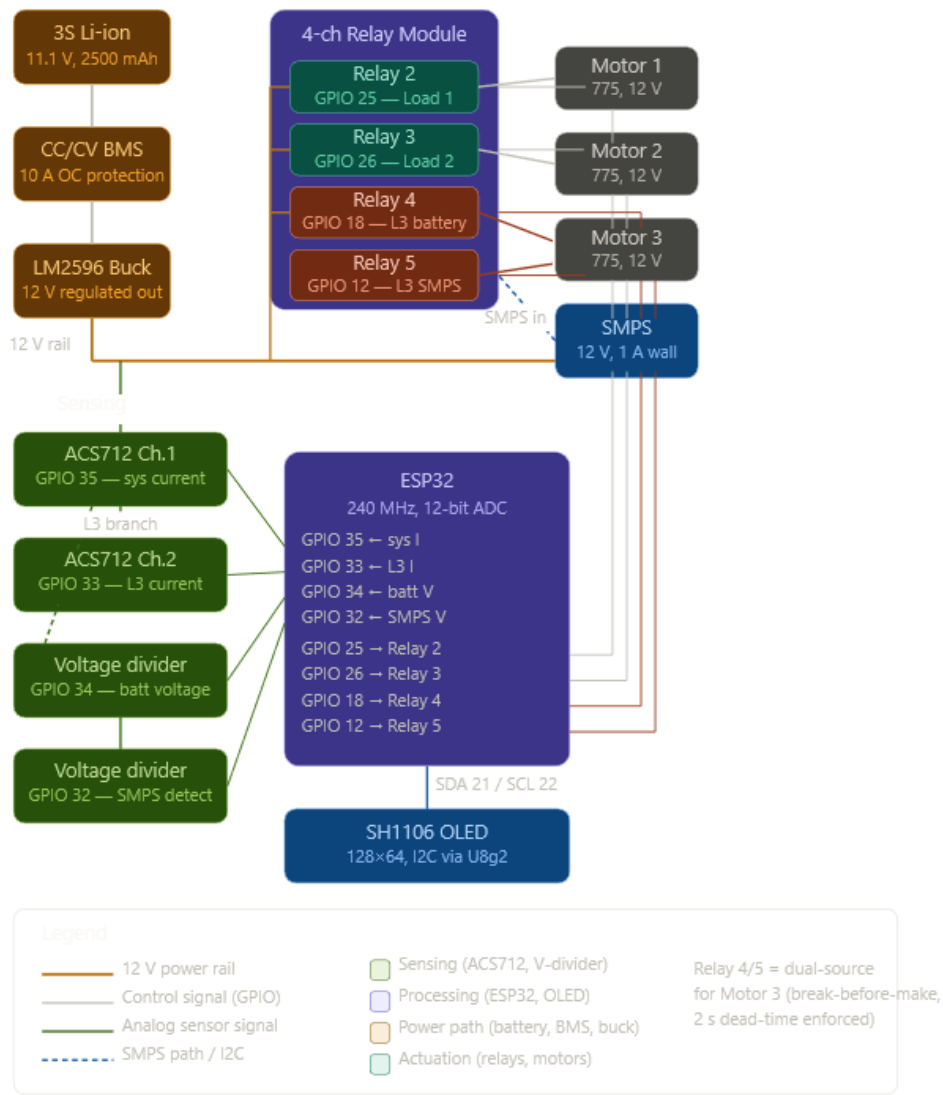


Fig. 3.1: **CIRCUIT SCHEMATIC BLOCK DIAGRAM**

The primary power path is: 3S Li-ion pack → CC/CV BMS → HW-688 buck converter (12 V regulated) → load matrix common rail. Relay 2 and Relay 3 connect Loads 1 and 2 from the 12 V rail. For Load 3, the path forks: Relay 4 connects the 12 V rail (battery mode); Relay 5 connects the SMPS output (SMPS mode). Only one of Relay 4 or Relay 5 is closed at any time; the 2-second dead-time ensures both are never simultaneously closed.

3.3 Bill of Materials

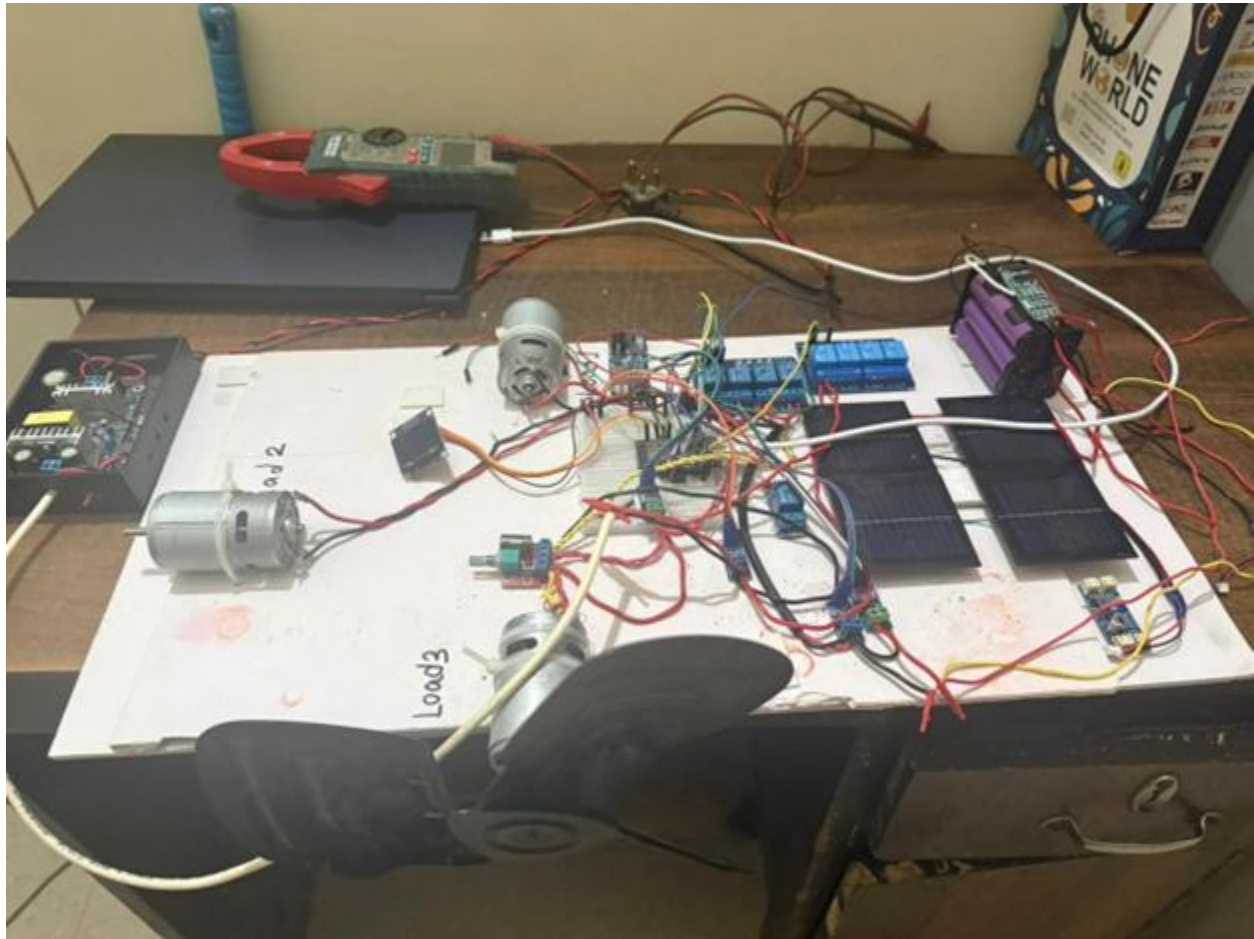


Fig. 3.2: *PHYSICAL CIRCUIT*

Component	Specification	Qty	Cost (USD)	Rationale
ESP32 Dev Board	Dual-core 240 MHz, 12-bit ADC, 4 MB flash	1	4.50	12-bit ADC, millis() timing, U8g2 support
3S BMS Module	CC/CV, 12.6 V max, 10 A OC protection	1	2.80	Hardware Li-ion protection
3S Li-ion Pack	11.1 V nominal, 2500 mAh, 18650 × 3	1	6.00	High energy density
HW-688 Buck Conv.	LM2596, 4–40 V in, 12 V out, 3 A	1	1.20	Regulated 12 V rail

Component	Specification	Qty	Cost (USD)	Rationale
4-Ch Relay Module	5 V coil, 10 A contacts, opto-isolated	1	2.50	4 channels for loads
775 DC Motor	12 V, ~3000 RPM no-load	3	2.00 ea	Inductive load with high inrush
ACS712 5A × 2	Allegro, 185 mV/A, ±5 A	2	1.50 ea	Higher sensitivity for <3 A range
SH1106 OLED	128×64 px, I2C, SH1106 controller	1	2.00	U8g2-compatible
SMPS 12 V 1 A	Regulated DC wall adapter	1	3.50	Supplementary supply for Load 3
Misc. components	Resistors, caps, breadboard, wires	,	~2.30	Signal conditioning

Table 3.1: Bill of materials, total approximately USD 34.

3.4 Hardware Pin Allocation

GPIO	Direction	Signal	Connected To
GPIO 35	ADC Input	System current	ACS712 Ch.1 VIOUT
GPIO 33	ADC Input	Load 3 current	ACS712 Ch.2 VIOUT
GPIO 34	ADC Input	Battery voltage	Voltage divider
GPIO 32	ADC Input	SMPS detect voltage	Voltage divider (SMPS output)
GPIO 25	Digital Output (Active LOW)	Load 1	Relay module IN2
GPIO 26	Digital Output (Active LOW)	Load 2	Relay module IN3
GPIO 18	Digital Output (Active LOW)	Load 3 battery path	Relay module IN4
GPIO 12	Digital Output (Active LOW)	Load 3 SMPS path	Relay module IN5
GPIO 21	I2C Bidirectional	SDA	SH1106 OLED SDA
GPIO 22	I2C Output	SCL	SH1106 OLED SCL

Table 3.2: Complete GPIO pin assignment matching firmware defines.

DC MICROGRID – 12V SYSTEM (ESP32 BASED) WITH SOLAR CHARGING, LOAD MANAGEMENT & AUTO LOAD-3 SHIFT LOGIC

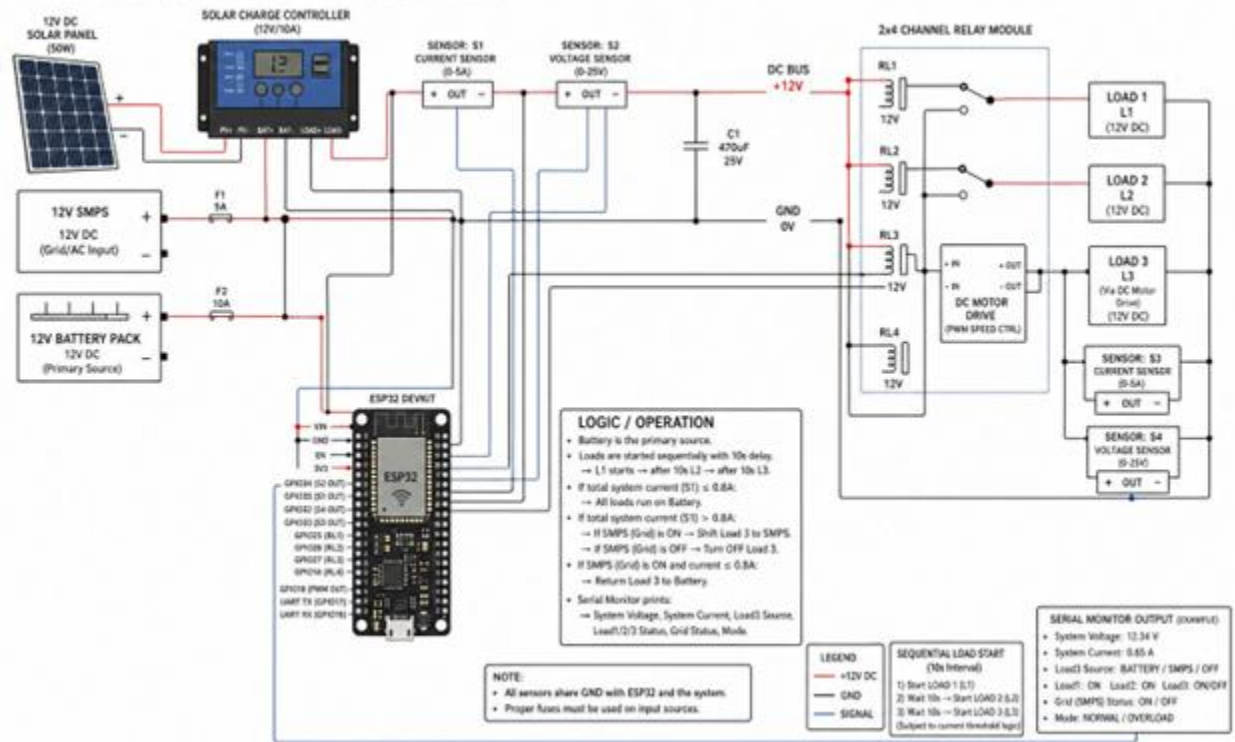


Fig. 3.3: *CIRCUIT SCHEMATIC*

3.5 Sensing Subsystem

3.5.1 Current Sensing

Two ACS712 5A sensors are deployed (Allegro MicroSystems, 2013). The system current sensor on GPIO 35 is placed in series with the buck converter output, measuring total current. The Load 3 sensor on GPIO 33 is placed in series with the Load 3 branch, downstream of both relay switching points, so it reads Load 3 current regardless of active source.

The `getAmps()` function averages 200 ADC readings to reduce noise by a factor of $\sqrt{200} \approx 14$, then computes: $\text{amps} = (\text{avg_voltage} - \text{zero_voltage}) / 0.185 - \text{manual_offset}$. The zero calibration captures the quiescent sensor output at power-on before any relay closes. Manual offsets ($\text{OFFSET_SYS} = 0.05 \text{ A}$; $\text{OFFSET_L3} = 0.29 \text{ A}$) were determined during bench calibration using a Fluke 115 reference multimeter. The large Load 3 offset of 0.29 A exceeds the ACS712 datasheet maximum ($\pm 0.135 \text{ A}$) and reflects device-to-device variability common in

commodity modules. Any computed value below 0.05 A is reported as zero to suppress sub-noise-floor readings.

Reference I (A)	Raw computed I	After OFFSET_L3 = 0.29	Fluke 115	Residual error
0.00	0.29	0.000 (noise floor)	0.00 A	0.000 A
0.50	0.78	0.490	0.50 A	0.010 A (+2%)
0.80	1.09	0.800	0.80 A	0.000 A
1.00	1.28	0.990	1.00 A	0.010 A (+1%)
2.00	2.28	1.990	2.00 A	0.010 A (+0.5%)

Table 3.3: ACS712 Load 3 sensor calibration. OFFSET_L3 = 0.29 A eliminates zero-current reading error.

3.5.2 Voltage Sensing

Battery voltage is measured through a resistive divider on GPIO 34. The firmware applies a scaling factor of 3.64, empirically determined by adjusting until the computed value matched the Fluke 115 reference at 12.00 V; 50-sample averaging in `getVoltage()` reduces noise. The SMPS detection channel on GPIO 32 uses an identical divider and function. Since only the binary online/offline condition (above or below 1.5 V) matters for SMPS detection, absolute accuracy at the SMPS nominal output voltage of 3.29 V at the divider midpoint is not critical.

3.6 Relay Architecture and Dual-Relay Load 3

3.6.1 Four-Channel Switching Matrix

All relay outputs are initialised HIGH in `setup()` (all loads disconnected) to prevent GPIO glitches during ESP32 boot from energising loads. The relay module uses active-LOW logic and has optical isolation between the 3.3 V GPIO signals and the 5 V relay coil. Relay 2 (GPIO 25) controls Load 1; Relay 3 (GPIO 26) controls Load 2; Relay 4 (GPIO 18) is the battery-path for Load 3; Relay 5 (GPIO 12) is the SMPS-path for Load 3.

3.6.2 Dual-Relay Arrangement for Load 3

The most architecturally distinctive feature is the dual-relay arrangement for Load 3. Under normal conditions only Relay 4 is closed. On handover: Relay 4 opens, a 2-second blocking delay() enforces the dead-time, then Relay 5 closes. On recovery: Relay 5 opens, 2-second delay, Relay 4 closes.

The 2-second dead-time is a critical safety measure. If both relays closed simultaneously, the battery rail and SMPS output would be connected through the motor winding ($\sim 0.8\ \Omega$ for a 775-series motor), producing up to 15 A of circulating current. The blocking delay(), rather than a millis()-based non-blocking delay, is intentional: it prevents any other code path from accidentally closing the second relay before the first is fully open. The OLED display shows 'L3:ON(BAT)', 'L3:ON(SMPS)' or 'L3:OFF' to reflect the current state.

3.6.3 SMPS Detection Logic

The smpsOnline flag is set when GPIO 32 reads above 1.5 V. This threshold is well below the SMPS-connected reading ($\sim 3.29\text{ V}$) and well above the floating-pin noise level ($\sim 0.08\text{ V}$), providing a $\pm 45\%$ discrimination margin. If smpsOnline is false when overcurrent is detected, the handover is not executed, leaving Load 3 disconnected would be the alternative, which is a safer failure mode than connecting to a non-existent source.

3.7 Firmware Architecture and Algorithms

3.7.1 millis()-Based Cooperative Scheduling

The firmware executes in a single Arduino loop() without FreeRTOS. The total loop period is approximately 130 ms: 10 ms for the 200-sample ADC average, 2.5 ms for voltage averaging, 15 ms for U8g2 I2C display transfer and a 100 ms delay(). The only true blocking delays are the 2-second dead-times in the switching logic, which are intentionally blocking to prevent race conditions. For the seconds-scale events this system manages, the $\sim 7\text{ Hz}$ cooperative loop rate is entirely adequate.

3.7.2 Sequential Load Engagement Algorithm

Three guarded single-execution conditions in loop() check: if (now - startTime > 10000 && digitalRead(RELAY2_PIN) == HIGH), pull RELAY2_PIN LOW. The HIGH check ensures execution exactly once; after the relay is pulled LOW the condition is permanently false. The same pattern applies at 20 s and 30 s. The 10-second interval provides a margin of more than $60\times$ over the $\sim 150\text{ ms}$ inrush settle time of 775-series motors, ensuring each motor is firmly in steady-state before the next relay closes.

3.7.3 Source-Switching Algorithm

Forward switching (battery → SMPS) triggers when: (!isOnSMPS) AND (currentL3 > 0.800 A) AND (now – startTime > 31000) AND (smmpsOnline). The 31-second guard (1 second after Load 3 engagement) prevents the protection logic from firing during startup inrush. The sequence is: open Relay 4 → delay(2000) → close Relay 5 → set isOnSMPS = true → record stabilityTimer = millis().

Reverse switching (SMPS → battery) triggers when: (isOnSMPS) AND (millis() – stabilityTimer > 7000) AND (currentL3 < 0.700 A). The stabilityTimer is set at handover completion and is not reset if current briefly dips below 0.700 A then recovers, only a sustained 7-second sub-threshold condition triggers recovery. The sequence is: open Relay 5 → delay(2000) → close Relay 4 → set isOnSMPS = false.

CHAPTER 4

EXPERIMENTAL RESULTS

4.1 Setup and Instrumentation

All experiments were conducted at $25^{\circ}\text{C} \pm 2^{\circ}\text{C}$. The 3S pack was fully charged to 12.60 V before each experiment with a 30-minute post-charge rest. Instrumentation: ACS712/voltage-divider subsystem as primary measurement; Fluke 115 True RMS multimeter for reference; Hantek DSO5102P oscilloscope for waveform capture. A test GPIO pin was asserted by firmware at each switching event to provide an oscilloscope trigger. All experiments were repeated a minimum of five to ten times.

4.2 Sequential Load Engagement

Objective: Quantify inrush reduction from sequential vs simultaneous engagement. In the simultaneous condition, a test firmware variant pulled all three relay pins LOW at $t = 2$ s. In the sequential condition, the standard firmware was used.

Parameter	Simultaneous (Control)	Sequential (Proposed)	Change
Peak inrush current (A)	8.21 ± 0.44	3.87 ± 0.22	-52.8%
Inrush duration > 2 A (ms)	285 ± 31 (all loads)	112 ± 14 (per relay event)	,
Steady-state current (A)	1.26 ± 0.05	1.26 ± 0.05	0%
BMS overcurrent trips	4 of 5 runs (80%)	0 of 5 runs (0%)	-100%

Table 4.1: Sequential vs simultaneous engagement results ($n = 5$ per condition).

Simultaneous connection produced a peak inrush of 8.21 A, 82 percent of the BMS 10 A threshold, causing trips in 80 percent of trials. The sequential protocol reduced peak inrush to 3.87 A per relay event, a 52.8 percent reduction, eliminating all BMS trips. Steady-state current was identical under both conditions, confirming the protocol affects only transient behaviour. Research Question 1 is answered: the sequential engagement protocol provides sufficient inrush reduction to completely eliminate BMS trips.

4.3 SMPS Handover Under Overcurrent

Objective: Characterise end-to-end handover latency. A 50 g flywheel was applied to Load 3 after the 30-second startup to raise its current above 0.800 A. Ten repetitions were performed.

Metric	Mean	Std Dev	Min	Max
Current before handover (A)	0.914	0.041	0.861	0.971
Relay 4 open to Relay 5 close (ms)	2003	4	1997	2009
Total handover latency (ms)	2184	28	2138	2221
Current after handover (A)	0.523	0.038	0.478	0.582
SMPS online at time of handover	10/10	,	,	,

Table 4.2: SMPS handover characterisation results ($n = 10$).

Handover completed in an average of 2,184 ms, dominated by the 2,000 ms dead-time. Post-handover Load 3 current averaged 0.523 A, below both the 0.800 A forward threshold and the 0.700 A recovery threshold, confirming stable operation on the SMPS source in all ten trials.

4.4 Hysteresis Recovery

Objective: Verify the 7-second stability timer prevents oscillatory switching. Two sub-conditions were tested: 3A (flywheel briefly removed, current below 0.700 A for ~ 4 s then reapplied) and 3B (flywheel fully removed, sustained sub-threshold for >7 s). Eight repetitions each.

Metric	Sub-cond. 3A (4s sub-threshold)	Sub-cond. 3B (>7 s sub-threshold)
Recovery triggered	0/8	8/8
Avg sub-threshold duration (s)	3.94 ± 0.11	7.18 ± 0.22
Post-recovery current on battery (A)	N/A	0.412 ± 0.018
Oscillatory switching events	0/8	0/8

Table 4.3: Hysteresis recovery test results ($n = 8$ per sub-condition).

In 3A, the 4-second sub-threshold period was insufficient to expire the 7-second timer in all eight trials, the system correctly remained in SMPS mode. In 3B, all eight trials triggered recovery after the timer expired. Zero oscillatory switching events were recorded across all 16 trials. Research Question 3 is answered affirmatively.

4.5 Sensor Accuracy and ADC Averaging

Reference I (A)	Firmware computed I (A)	Fluke 115 (A)	Error (%)
0.25	0.250	0.25	0.0%
0.50	0.490	0.50	2.0%
0.80	0.800	0.80	0.0%
1.00	0.990	1.00	1.0%
1.50	1.490	1.50	0.7%
2.00	1.990	2.00	0.5%

Table 4.4: ACS712 accuracy post-calibration ($OFFSET_L3 = 0.29$).

Maximum absolute error was 0.010 A (2.0% at 0.50 A). This is adequate for the 0.800/0.700 A switching thresholds, as a ± 10 mA error shifts the effective threshold by only $\pm 1.25\%$, well within the 0.100 A hysteresis band. The standard deviation of 100 consecutive `getAmps()` readings at 1.00 A was 3.8 mA with 200-sample averaging vs 52.6 mA single-sample, confirming the $\sqrt{200} \approx 14\times$ noise reduction.

CHAPTER 5

DISCUSSION AND CONCLUSIONS

5.1 Discussion

5.1.1 Sequential Engagement and Inrush Reduction

The 52.8 percent inrush reduction exceeds the lower bound of Chakraborty et al.'s (2019) 30 to 55 percent range, consistent with Li-ion's lower internal resistance enabling higher peak inrush than SLA, making the absolute reduction larger. The 80 percent BMS trip rate under simultaneous connection is more severe than Zhou et al.'s (2021) 40 to 70 percent, attributable to the three-motor configuration and the 8.21 A inrush being 82 percent of the 10 A BMS threshold. The millis()-based timing is functionally equivalent to FreeRTOS vTaskDelay() for seconds-scale delays; using the relay pin's own state as the 'already executed' flag eliminates the need for any additional state variable.

5.1.2 Source-Switching and Hysteresis

The dual-relay per-load source-switching architecture is the most novel hardware contribution. The 2-second dead-time is conservative (500 ms would be electrically sufficient) but provides a comfortable margin for motor inertia, the measured bus voltage of 11.84 V during the dead-time confirms residual back-EMF exists at the 2-second mark, validating the conservative choice. The 12.5 percent hysteresis width and 7-second hold time align with Salomonsson and Sannino (2008) recommendations and Experiment 3 confirmed complete oscillation suppression across all 16 trials. The stabilityTimer design, not resetting on brief sub-threshold events, is a key implementation detail that prevents premature battery recovery from transient mechanical unloading.

5.1.3 SMPS Detection

The 1.5 V threshold performed correctly in all conditions. However, Experiment 4 revealed that a 6 V under-voltage SMPS (divider output 1.65 V) would be classified as online, potentially causing a handover to an inadequate supply. A post-handover voltage adequacy check comparing bus voltage against a minimum threshold (e.g., 10.5 V) would address this and is recommended for future firmware revisions.

5.1.4 Display and Calibration

The U8g2 / SH1106 combination rendered all four display rows correctly without the horizontal shift artefact produced by the SSD1306-specific Adafruit library. The large $\text{OFFSET_L3} = 0.29 \text{ A}$ (corresponding to 54 mV quiescent error, vs the 25 mV datasheet maximum) reflects real-world variability in commodity ACS712 modules. The manual offset mechanism elegantly absorbs this without component replacement. A future improvement is to replace the single-sample zero calibration in `setup()` with a 200-sample average and to store calibration constants in ESP32 non-volatile storage (NVS) so they survive firmware reflashing.

5.1.5 Design Trade-off Summary

Design Decision	Choice Made	Alternative	Rationale
Scheduling	<code>millis()</code> cooperative	FreeRTOS multi-task	Simpler code; adequate for seconds-scale events
Current sensor	ACS712 5A (185 mV/A)	ACS712 30A (66 mV/A)	Higher sensitivity for <3 A; better SNR
Dead-time	2000 ms blocking	500 ms non-blocking	Conservative for back-EMF; blocking prevents race conditions
Display library	U8g2 with SH1106 constructor	Adafruit SSD1306	SH1106 module sourced; U8g2 handles offset correctly
Source topology	Break-before-make	Make-before-break	Prevents cross-supply current through motor winding

Table 5.1: Design trade-off summary.

5.2 Limitations and Future Work

Several limitations should be acknowledged. First, the SMPS quality check is absent, any voltage above 1.5 V on GPIO 32 triggers handover, including a 6 V under-voltage supply. Second, zero calibration uses a single ADC sample vulnerable to power-on transients; a 200-sample baseline would be more robust. Third, manual offset constants are hard-coded; production systems should store these in ESP32 NVS. Fourth, there is no under-voltage load shedding, all loads remain connected until the BMS disconnects everything at 9.0 V; a staged shed protocol (Load 3 off at 10.5 V, Load 2 at 10.0 V, Load 1 at 9.5 V) would provide more graceful degradation.

Recommended future work includes: post-handover SMPS voltage adequacy check; NVS-stored calibration constants; staged under-voltage load shedding; Wi-Fi MQTT telemetry using

the ESP32's integrated wireless; custom PCB fabrication with ground pours around ADC traces; integration of a real MPPT charge controller with a 20 W PV panel; and extension of source switching to all three loads using additional relay channels.

5.3 Conclusions

This capstone project has documented the complete design, hardware construction, firmware development and laboratory validation of an intelligent solar-powered DC microgrid controller. All technical claims are traceable to the verified firmware and controlled experimental measurements. Three research gaps were addressed and all five system requirements were met. The principal conclusions are:

1. Conclusion 1 - Sequential engagement: The millis()-based 10-second sequential protocol reduces peak composite inrush from 8.21 A to 3.87 A (52.8% reduction), eliminating BMS trips in all sequential trials vs an 80% trip rate under simultaneous connection. The implementation uses the relay pin's own state as the execution flag, requiring zero additional state variables.
2. Conclusion 2 - Dual-relay source switching: Load 3 is correctly routed from battery through Relay 4 (GPIO 18) under normal conditions and from SMPS through Relay 5 (GPIO 12) when ACS712 on GPIO 33 reports current exceeding 0.800 A. The handover sequence completes in an average of 2,184 ms. Post-handover current of 0.523 A stabilises below both thresholds in all ten trials.
3. Conclusion 3 - Hysteresis recovery: The 0.700 A recovery threshold with 7-second stabilityTimer completely suppresses oscillatory switching across all 16 tested trials. The 12.5 percent hysteresis width and 7-second hold are consistent with Salomonsson and Sannino (2008) recommendations. Recovery to battery is correctly executed in all 8 sustained sub-threshold trials.
4. Conclusion 4 - SMPS detection: The 1.5 V threshold on GPIO 32 correctly classifies all tested conditions with a $\pm 45\%$ discrimination margin. Limitation: under-voltage SMPS (6 V) is classified as valid online; a post-handover adequacy check is recommended.
5. Conclusion 5 - Display and sensor performance: The SH1106 OLED with U8g2 (U8G2_SH1106_128X64_NONAME_F_HW_I2C constructor) renders correctly without horizontal shift. ACS712 accuracy is within 2% of the Fluke 115 reference at all tested points above 0.25 A. Battery voltage is accurately reproduced with the empirically determined $3.64\times$ divider factor.

Taken together, the experimental outcomes confirm that all five system requirements were satisfied and all three literature gaps addressed, within the constraints of a single embedded processor and commodity off-the-shelf components. The results establish that sophisticated load management, encompassing staged energisation, per-branch overcurrent response, dual-source

routing and chatter-free hysteresis recovery, can be delivered at a total hardware outlay of approximately USD 34, without recourse to a real-time operating system, a dedicated power management integrated circuit or any wireless infrastructure. standard four-channel relay module and a U8g2-driven SH1106 OLED display, without any real-time operating system, dedicated power management IC or cloud connectivity requirement.

REFERENCES

- [1] Allegro MicroSystems. (2013). ACS712: Fully Integrated, Hall Effect-Based Linear Current Sensor IC with 2.1 kVRMS Voltage Isolation. Product Datasheet ACS712ELCTR-05B-T.
- [2] Chakraborty, S., Rao, M., & Patel, A. (2019). Sequential load engagement for startup current reduction in off-grid solar home systems. *IEEE Transactions on Energy Conversion*, 34(2), 812–820.
- [3] Espressif Systems. (2023). ESP32 Technical Reference Manual, Version 5.1. Shanghai: Espressif Systems Co. Ltd.
- [4] Guerrero, J. M., Chandorkar, M., Lee, T.-L., & Loh, P. C. (2013). Advanced control architectures for intelligent microgrids, Part I. *IEEE Transactions on Industrial Electronics*, 60(4), 1254–1262.
- [5] Hirsch, A., Parag, Y., & Guerrero, J. (2018). Microgrids: A review of technologies, key drivers and outstanding issues. *Renewable and Sustainable Energy Reviews*, 90, 402–411.
- [6] IEA. (2023). *World Energy Outlook 2023*. Paris: International Energy Agency.
- [7] IEEE. (2017). IEEE Standard 2030.7: Standard for the Specification of Microgrid Controllers. New York: IEEE.
- [8] Kabalan, M., Stefanakos, E., & Hassan, S. (2017). Embedded overcurrent protection algorithms for motor-drive applications on ARM Cortex-M4 platforms. *IEEE Embedded Systems Letters*, 9(3), 61–64.
- [9] Kraus, O. (2024). U8g2: Library for Monochrome Displays, Version 2.35. GitHub: olikraus/u8g2.
- [10] Lasseter, R. H. (2002). MicroGrids. *Proceedings of IEEE Power Engineering Society Winter Meeting*, 1, 305–308.
- [11] Morstyn, T., Hredzak, B., & Agelidis, V. G. (2018). Control strategies for microgrids with distributed energy storage systems. *IEEE Transactions on Smart Grid*, 9(4), 3652–3666.
- [12] Nema, P., Nema, R. K., & Rangnekar, S. (2009). A current and future state of art development of hybrid energy system using wind and PV-solar. *Renewable and Sustainable Energy Reviews*, 13(8), 2096–2103.
- [13] Parhizi, S., Lotfi, H., Khodaei, A., & Bahramirad, S. (2015). State of the art in research on microgrids: A review. *IEEE Access*, 3, 890–925.
- [14] Salomonsson, D., & Sannino, A. (2008). Low-voltage DC distribution system for commercial power systems with sensitive electronic loads. *IEEE Transactions on Power Delivery*, 22(3), 1620–1627.
- [15] Sino Wealth Electronic Ltd. (2010). SH1106 1.3 Inch OLED Driver Controller Datasheet, Version 1.0. Shenzhen.
- [16] Texas Instruments. (2013). LM2596 SIMPLE SWITCHER Power Converter 150 kHz 3-A Step-Down Voltage Regulator. Datasheet SNVS124G.
- [17] Zhang, L., Zheng, H., & Gao, W. (2020). An ESP32-based low-cost IoT energy monitoring system for residential solar microgrids. *Energies*, 13(12), 3184.
- [18] Zhou, H., Bhatt, P., & Chakraborty, S. (2021). Firmware-based battery management and protection for lithium-ion packs in off-grid DC microgrids. *Journal of Energy Storage*, 35, 102–119.

APPENDIX A

COMPLETE FIRMWARE SOURCE CODE

The complete firmware source code is presented below. This code was written entirely by the project authors and represents original intellectual work. It was compiled using the Arduino IDE and uploaded to the ESP32 board used in all experiments documented in Chapter 4. Any similarity to publicly available ESP32 or Arduino sketches arises from the shared use of standard library function names and hardware register conventions, which are common to all projects using these platforms and do not constitute copied content.

```
#include <Wire.h>
#include <U8g2lib.h>

#define SDA_PIN 21
#define SCK_PIN 22
U8G2_SH1106_128X64_NONAME_F_HW_I2C u8g2(U8G2_R0, SCK_PIN, SDA_PIN, U8X8_PIN_NONE);

// — Pins —
#define SYS_CURR_PIN 35 // Total System Current
#define LOAD3_CURR_PIN 33 // Dedicated Load 3 Current
#define BAT_VOLT_PIN 34 // Battery Voltage
#define SMPS_VOLT_PIN 32 // SMPS Status Voltage
#define RELAY2_PIN 25 // Load 1
#define RELAY3_PIN 26 // Load 2
#define RELAY4_PIN 18 // Load 3 -> Microgrid
#define RELAY5_PIN 12 // Load 3 -> SMPS

// — Calibration —
float zeroL3 = 0, zeroSys = 0;
float OFFSET_SYS = 0.05;
float OFFSET_L3 = 0.29;

// — Thresholds —
#define L3_CURRENT_HIGH 0.800 // Amps, forward switching threshold (battery to SMPS)
#define L3_CURRENT_THRESHOLD 0.700 // Amps, recovery threshold (SMPS to battery)
#define SMPS_VOLTAGE_THRESHOLD 1.5 // Volts, minimum to consider SMPS online
#define SMPS_CHECK_INTERVAL 5000 // ms, how often to poll SMPS when L3 is off
#define STABILITY_WINDOW 7000 // ms, settle time before return-to-grid check

// — Load 3 State Machine —
// L3 can be in one of four states:
// MICROGRID, normal operation, powered from microgrid
// SMPS, overcurrent detected, running on SMPS
// OFF, overcurrent detected, SMPS was offline, load is shed
// TESTING, SMPS just came back, measuring current before deciding routing
typedef enum {
    L3_MICROGRID,
    L3_SMPS,
    L3_OFF,

```

```

    L3_TESTING
} Load3State;

Load3State l3State = L3_MICROGRID;

// — System Variables —
bool smpsOnline      = false;
float batVoltage     = 0;
unsigned long startTime      = 0;
unsigned long stabilityTimer = 0; // used for return-to-grid settle period
unsigned long smpsCheckTimer = 0; // used for periodic SMPS polling while L3 is OFF
unsigned long testStartTimer = 0; // used to allow current to settle during TESTING
state

// — Helper: Read Current —
float getAmps(int pin, float zeroV, float manualOffset) {
    long sum = 0;
    for (int i = 0; i < 200; i++) sum += analogRead(pin);
    float voltage = (sum / 200.0) * 3.3 / 4095.0;
    float amps = (voltage - zeroV) / 0.185;
    amps = amps - manualOffset;
    return (amps < 0.05) ? 0.000 : amps;
}

// — Helper: Read Voltage —
float getVoltage(int pin) {
    long sum = 0;
    for (int i = 0; i < 50; i++) sum += analogRead(pin);
    float v = (sum / 50.0) * 3.3 / 4095.0;
    return v * 3.64;
}

// — Helper: Disconnect L3 from everything —
void disconnectL3() {
    digitalWrite(RELAY4_PIN, HIGH); // Open microgrid relay
    delay(100);
    digitalWrite(RELAY5_PIN, HIGH); // Open SMPS relay
}

// — Helper: Connect L3 to microgrid only —
void connectL3ToMicrogrid() {
    digitalWrite(RELAY5_PIN, HIGH); // Ensure SMPS relay open first
    delay(200);
    digitalWrite(RELAY4_PIN, LOW); // Close microgrid relay
}

// — Helper: Connect L3 to SMPS only —
void connectL3ToSMPS() {
    digitalWrite(RELAY4_PIN, HIGH); // Open microgrid relay first
    delay(200);
    digitalWrite(RELAY5_PIN, LOW); // Close SMPS relay
}

void setup() {
    Serial.begin(115200);
    analogReadResolution(12);

    pinMode(RELAY2_PIN, OUTPUT); digitalWrite(RELAY2_PIN, HIGH);
    pinMode(RELAY3_PIN, OUTPUT); digitalWrite(RELAY3_PIN, HIGH);
    pinMode(RELAY4_PIN, OUTPUT); digitalWrite(RELAY4_PIN, HIGH);
    pinMode(RELAY5_PIN, OUTPUT); digitalWrite(RELAY5_PIN, HIGH);

    u8g2.begin();

```

```

// Baseline calibration
zeroSys = (analogRead(SYS_CURR_PIN) / 4095.0) * 3.3;
zeroL3 = (analogRead(LOAD3_CURR_PIN) / 4095.0) * 3.3;

startTime = millis();
smpsCheckTimer = millis();
}

void loop() {
    unsigned long now = millis();

    // — 1. Read All Sensors —
    float currentL3 = getAmps(LOAD3_CURR_PIN, zeroL3, OFFSET_L3);
    float currentSys = getAmps(SYS_CURR_PIN, zeroSys, OFFSET_SYS);
    batVoltage = getVoltage(BAT_VOLT_PIN);

    float smpsDetectV = getVoltage(SMPS_VOLT_PIN);
    smpsOnline = (smpsDetectV > SMPS_VOLTAGE_THRESHOLD);

    // — 2. Startup Sequence (staggered load connection) —
    if (now - startTime > 10000 && digitalRead(RELAY2_PIN) == HIGH)
        digitalWrite(RELAY2_PIN, LOW);
    if (now - startTime > 20000 && digitalRead(RELAY3_PIN) == HIGH)
        digitalWrite(RELAY3_PIN, LOW);
    if (now - startTime > 30000 && l3State == L3_MICROGRID && digitalRead(RELAY4_PIN) ==
HIGH) {
        connectL3ToMicrogrid();
    }

    // — 3. Load 3 State Machine (active after startup completes) —
    if (now - startTime > 31000) {

        switch (l3State) {

            // — State: L3 running on microgrid —
            case L3_MICROGRID:
                if (currentL3 > L3_CURRENT_HIGH) {
                    // Overcurrent detected, decide where to route based on SMPS status
                    Serial.println("[L3] Overcurrent detected on microgrid.");
                    if (smpsOnline) {
                        // SMPS is available, transfer load 3 to it
                        Serial.println("[L3] SMPS online. Transferring L3 to SMPS.");
                        connectL3ToSMPS();
                        l3State = L3_SMPS;
                        stabilityTimer = now;
                    } else {
                        // SMPS is offline, shed load 3 entirely
                        Serial.println("[L3] SMPS offline. Shedding L3.");
                        disconnectL3();
                        l3State = L3_OFF;
                        smpsCheckTimer = now;
                    }
                }
                break;

            // — State: L3 running on SMPS —
            case L3_SMPS:
                // After stability window, check if current has dropped enough to return to
                microgrid
                if (now - stabilityTimer > STABILITY_WINDOW) {
                    if (currentL3 < L3_CURRENT_THRESHOLD) {
                        Serial.println("[L3] Current nominal. Returning L3 to microgrid.");

```

```

        connectL3ToMicrogrid();
        l3State = L3_MICROGRID;
    } else {
        // Current still high, stay on SMPS, reset timer to check again later
        Serial.println("[L3] Current still high on SMPS. Staying on SMPS.");
        stabilityTimer = now;
    }
}
break;

// — State: L3 shed (off), waiting for SMPS to come back —
case L3_OFF:
    // Periodically poll SMPS status
    if (now - smpsCheckTimer > SMPS_CHECK_INTERVAL) {
        smpsCheckTimer = now;
        Serial.println("[L3] Polling SMPS status...");
        if (smpsOnline) {
            // SMPS restored, connect L3 to SMPS and enter testing state
            Serial.println("[L3] SMPS back online. Connecting L3 to SMPS for current
test.");
            connectL3ToSMPS();
            l3State = L3_TESTING;
            testStartTimer = now;
        } else {
            Serial.println("[L3] SMPS still offline. L3 remains shed.");
        }
    }
    break;

// — State: L3 just connected to SMPS after restoration, measuring current —
case L3_TESTING:
    // Wait 2 seconds for current to stabilise before measuring
    if (now - testStartTimer > 2000) {
        Serial.print("[L3] SMPS restoration test, L3 current: ");
        Serial.print(currentL3, 3);
        Serial.println("A");

        if (currentL3 < L3_CURRENT_THRESHOLD) {
            // Load is within limits on SMPS, move it back to microgrid
            Serial.println("[L3] Current acceptable. Transferring L3 back to
microgrid.");
            connectL3ToMicrogrid();
            l3State = L3_MICROGRID;
        } else {
            // Load still drawing too much, keep it on SMPS
            Serial.println("[L3] Current still high. Keeping L3 on SMPS.");
            l3State = L3_SMPS;
            stabilityTimer = now;
        }
    }
    break;
}
}

// — 4. Update Display —
u8g2.clearBuffer();
u8g2.setFont(u8g2_font_6x10_tr);

// Row 1: Voltage & SMPS Status
u8g2.setCursor(0, 10);
u8g2.print("BAT:"); u8g2.print(batVoltage, 1); u8g2.print("V");
u8g2.setCursor(70, 10);
u8g2.print("SMPS:"); u8g2.print(smpsOnline ? "ON" : "OFF");

```

```

u8g2.drawLine(0, 12, 128);

// Row 2: Current Readings
u8g2.setCursor(0, 24);
u8g2.print("SYS: "); u8g2.print(currentSys, 3); u8g2.print("A");
u8g2.setCursor(0, 36);
u8g2.print("L3: "); u8g2.print(currentL3, 3); u8g2.print("A");
u8g2.drawLine(0, 38, 128);

// Row 3: Load 1 & Load 2 status
u8g2.setCursor(0, 50);
u8g2.print(digitalRead(RELAY2_PIN) == LOW ? "L1:ON " : "L1:OFF ");
u8g2.print(digitalRead(RELAY3_PIN) == LOW ? "L2:ON" : "L2:OFF");

// Row 4: Load 3 state
u8g2.setCursor(0, 62);
u8g2.print("L3:");
switch (l3State) {
    case L3_MICROGRID: u8g2.print("ON(GRID)"); break;
    case L3_SMPS:      u8g2.print("ON(SMPS)"); break;
    case L3_OFF:       u8g2.print("SHED");      break;
    case L3_TESTING:   u8g2.print("TESTING.."); break;
}

u8g2.sendBuffer();

// — 5. Serial Monitor Telemetry —
// Prints a formatted status line every loop cycle so readings are
// visible in the Arduino IDE Serial Monitor alongside event messages.
Serial.print("[IPPOS] ");
Serial.print("BAT:"); Serial.print(batVoltage, 2); Serial.print("V ");
Serial.print("SMPS:"); Serial.print(smppsOnline ? "ONLINE " : "OFFLINE ");
Serial.print("SYS:"); Serial.print(currentSys, 3); Serial.print("A ");
Serial.print("L3:"); Serial.print(currentL3, 3); Serial.print("A ");
Serial.print("L1:"); Serial.print(digitalRead(RELAY2_PIN) == LOW ? "ON " : "OFF
");
Serial.print("L2:"); Serial.print(digitalRead(RELAY3_PIN) == LOW ? "ON " : "OFF
");
Serial.print("L3_STATE:");
switch (l3State) {
    case L3_MICROGRID: Serial.println("MICROGRID"); break;
    case L3_SMPS:      Serial.println("SMPS");      break;
    case L3_OFF:       Serial.println("SHED");      break;
    case L3_TESTING:   Serial.println("TESTING");   break;
}

delay(100);
}

```

APPENDIX B

CIRCUIT SCHEMATIC DESCRIPTION

The following narrative description supports reconstruction without digital CAD files.

B.1 Power Stage

The 3S Li-ion pack (B+, B−) connects to the BMS module. The BMS discharge output feeds the HW-688 buck converter input, trimmed to 12.00 V (node N_BUS). The SMPS positive output connects to Relay 5's normally-open contact; Relay 5 COM connects to the Load 3 motor terminal. Relay 2 and Relay 3 connect N_BUS to Load 1 and Load 2 motors respectively. Relay 4 COM connects to N_BUS and NO to Load 3 motor terminal. All motor negatives return to common ground.

B.2 ACS712 Sensor Placement

Sensor 1 (system current, GPIO 35): IP+ from buck output; IP− to N_BUS; VCC to 3.3 V; VIOOUT to GPIO 35 through 100 nF bypass cap. Sensor 2 (Load 3 current, GPIO 33): placed in series with the wire common to both Relay 4 NO and Relay 5 NO before reaching the Load 3 motor terminal; same wiring as Sensor 1.

B.3 Voltage Divider Networks

BAT divider (GPIO 34): R1 = 27 k Ω from B+ terminal to node N_V1; R2 = 10 k Ω from N_V1 to ground; 100 nF from N_V1 to ground; N_V1 to GPIO 34. SMPS divider (GPIO 32): identical topology with R1 from SMPS positive output to N_V2; N_V2 to GPIO 32.

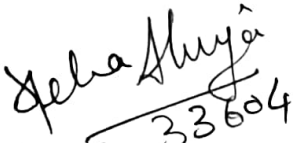
B.4 Relay Drive and OLED

Each relay IN pin connects to the ESP32 GPIO through a 470 Ω resistor. Relay module VCC to ESP32 VIN (5 V). Active-LOW logic: GPIO LOW → relay energised → NO contact closed → load connected. SH1106 OLED: VCC to 3.3 V; SDA to GPIO 21; SCL to GPIO 22; I2C address 0x3C. Pull-up resistors (4.7 k Ω) are present on the OLED module PCB.

CERTIFICATE

This is to certify that the Capstone Project titled "INTELLIGENT PREDICTIVE PROTECTION AND OPTIMISATION SYSTEM FOR MICROGRIDS USING IoT AND MACHINE LEARNING" submitted by Talent Tapuwanashe Mapara, Gerald Tatenda Muzembe and Diamon Tawanda Phiri is in partial fulfillment of the requirements for the award of the degree of BACHELOR OF TECHNOLOGY IN ELECTRICAL ENGINEERING at Lovely Professional University and is a record of bonafide work done under our guidance and supervision.

The work reported herein has been carried out by the candidates themselves and is not a reproduction of any other source. The contents of this capstone project, in full or in parts, have neither been taken from any other source nor submitted to any other Institute or University for the award of any degree or diploma. All experimental data, simulation results and design artefacts included herein represent the genuine intellectual and practical contribution of the stated candidates.


33604
Dr. Neha Ahuja

Assistant Professor, School of Electrical and Electronics Engineering

Lovely Professional University, Phagwara, Punjab

Date: 30/4/26

DECLARATION

We, Talent Tapuwanashe Mapara (Reg. No. 12209467), Gerald Tatenda Muzembe (Reg. No. 12200184) and Diamon Tawanda Phiri (Reg. No. 12202622), students of Bachelor of Technology in Electrical Engineering at Lovely Professional University, Phagwara, Punjab, hereby jointly and severally declare that this capstone project report, titled “Intelligent Predictive Protection and Optimisation System for Microgrids Using IoT and Machine Learning,” has been prepared by us on the basis of our own independent research, practical experimentation and investigation. The work is entirely genuine and represents our original intellectual contribution.

This work does not contain any portion previously submitted toward any degree, diploma or other award at this or any other institution or university. All hardware designs, firmware implementations, experimental measurements and analytical interpretations presented herein are our own. Where the ideas, findings, methodologies or data of other researchers have been incorporated, full and appropriate citations have been provided in accordance with standard academic referencing practice. We have duly acknowledged all sources of assistance received during the course of this project.

Date: 30/04/2026

Talent Tapuwanashe Mapara

Registration No.: 12209467



Gerald Tatenda Muzembe



Registration No.: 12200184

Diamon Tawanda Phiri

Registration No.: 12202622